Scotland yard report

Ss24495 & er24668

Throughout the coursework, we have produced a working build of the code that has passed all of the internal tests provided. We did not make an attempt on the CW-AI part of the coursework, as we felt that the time investment it would have required would not have brought an equivalent in marks given for it. Spending a greater amount of time on the CW-MODEL aspect was a wiser decision.

Our paired approach to this coursework was to coordinate what subclasses and methods we were focusing on in the work to each other. This was to make sure that neither of us pushed a git commit and either overwrote what the other person has written or spend a long time having to comb through a merging request in the code, outlining every line of code that was to be deleted and what was to be kept. When it got to parts of the coursework that were more challenging, we focused on them with the 2-person 1-computer approach in the dedicated labs sessions, proving to be helpful in both of us giving our inputs into an approach and covering for each other's mishaps that may have occurred.

One issue occurred from this when implementing the advance and move methods during the coding process. One of us had implemented partway through the advance method while the other was starting the singleMoves method. When working through singleMoves, advance had to be altered because the logic was incorrect and did not allow singleMoves to be correctly made. The person who originally started on advance forgot to git pull before starting again so when it came to committing their work, a huge portion of the code was receiving overwrite requests and it had to be all selected as to which parts were kept and removed. This was a setback in the work, but further emphasised the importance of communication between each other and making sure that your code was fully up to date before starting any work as to prevent any accidental branching.

As to our understanding, the game works off of a basis where each game state is immutable and therefore aspects of each game state cannot be changed or altered when created. This means that for every move or instance where the game would have to change in some way, a new game instance must be created and implemented to reflect any in-game changes. This was different to previous games that we may have created, but this approach keeps each representation of the game state extremely well defined and robust. By keeping aspects such as the moves, pieces remaining and log entries as immutable, it meant that these could not be altered by accident by stray

code. It also means that any intermediate states that are created – for example in doubleMoves – are not generated into the game until it is sure that it was a legal operation. This ideology demonstrates a focus on safety and ease of reasoning within the code, avoids accidental bugs and ensures a constant state.

Throughout the code, we made sure to implement conditional checks and validators plentifully. This was because as the game state is immutable, it was critical to make sure that each game state created was valid and there was no invalid game states produced during the game at all. Performing several null and illegal argument checks ensures that no aspect of the game shared any illegal aspects or were initialised empty.

A key concept of modularity also was pertinent to this coursework. For example, by offsetting the computing of moves to outside of the game state's update logic, it allowed more careful consideration into the declarations inside these helper methods and to allows the use of nested iteration and streaming of other methods into itself to reduce code complexity and readability, as well as code quality.

A big achievement to us was our implementation of advance and the visitor pattern. By 'visiting' the move with Move.Visitor<GameState>, the visitor pattern allows the program to handle each type of move separately from the move itself, and gives the option of handling both single and double moves differently, cleanly separating them and keeping the visitor maintainable. It eliminates the need of explicit casting and showcases a use of encapsulation.

By extension, another big achievement was the makeSingleMove and makeDoubleMove helper methods. Differentating between single and double moves was initially troublesome due to the special double ticket required, but after spending some time experimenting and studying with the ticket API it became clear the association of tickets within the pieces. Initially we had used streaming to create the new sets for single and double moves, but we found that just creating a new hash set was more efficient and created code that was easier to read. Navigating through graphs also proved to be quite difficult, with us ending up choosing the approach to iterate over all adjacent nodes to the current location of the piece, using the board graph from the setup. To check if the node was unoccupied by a detective, we at first tried to iteratively check every pieces location against the nodes found and then return whether the node was empty using a flag system. However, the flag ended up either returning wrong results or invalid errors, so we had to place a break statement. We were hesitant to create a break statement into our code but we couldn't think of a solution that wouldn't require us rewriting a lot of our previous logic so we decided to code it in anyway.

A critical point of our code that was hard to code was the observer interface. We had to wrap the game state in a model object and then return any state changes accurately to observers. We initially passed the first few tests, but then had big issues with one test that wouldn't return correctly to the observer. Our model kept returning an incorrect winner to the observer, one of them specifically when RED was supposed to be the winner and MRX was returned, or vice versa. We kept trying to alter our logic to the MyModelFactory code to try and remedy this issue, but to no avail. Ultimately, the issue was our winner logic within MyGameStateFactory, and we had to alter it's logic to provide a correct result to the observer. By removing some conditional statements that were producing erroneous results and introducing the winner statement of winner being defined as a stream of the players filtered to not MRX if a detective lands on MRX's piece, we were able to fix this issue and pass all of the tests.

A limitation of our coursework is the lack of any AI implementation into the code. This proved to be because of time constraints and inability to create a stable version within a reasonable time frame. I think our time management was a factor to blame for this, as I think we underestimated the task we had to complete, as well not enough of our time coding spent physically together. If we were to do this project again, I think we would arrange for more in-person sessions as it did feel limiting coding individually for a larger-than-necessary proportion of the project.